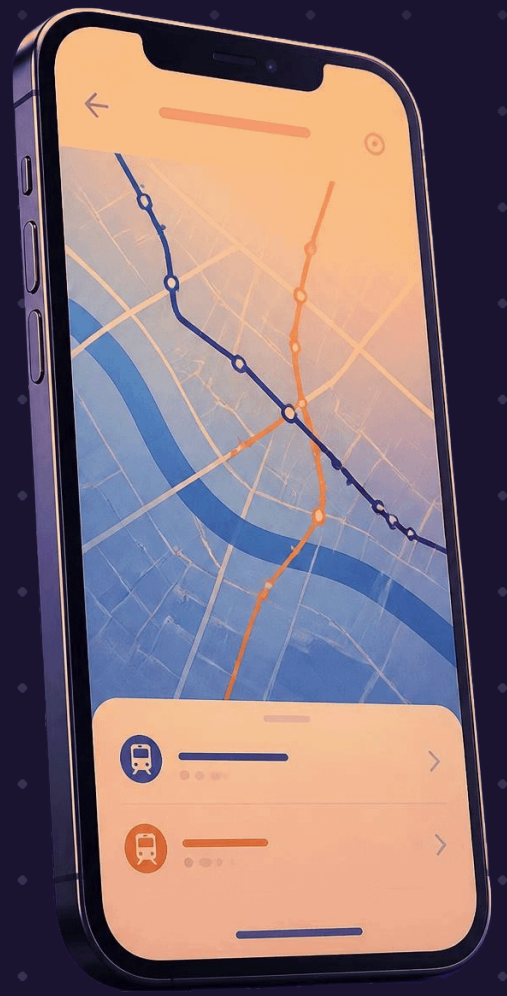
ATeMto

Tu agente asistencial para moverte por la red ATM

Autor: Raúl Molina Ordóñez

Multiagente · Recorridos · Tarifas · Próximo tren TMB



Made with GAMMA

Tres capacidades en una sola conversación y en un único plan

01 RECORRIDOS

Trayecto exacto

Calcula la ruta entre dos poblaciones con transbordos y operadores integrados.

02 TARIFAS

Abono óptimo

Recomienda el billete adecuado según tus zonas, uso y precio actualizado.

03 TIEMPO REAL

Próximo tren

Consulta la llegada inmediata del convoy en estaciones de la red TMB.

Caso de uso: Una persona en tiempo real podrá indicar donde está donde quiere ir y en un solo plan obtendrá el trayecto dentro de la red ATM, el mejor abono a adquirir y cuando llegará el próximo tren si la parada origen es de ATM.

Todo resuelto desde un único mensaje sin cambiar de aplicación ni consultar tablas PDF.

Autor: Raúl Molina Ordóñez

Made with GAMMA

Dominio: Tres herramientas dan base al agente

PLANO ATM

Multimodal

El LLM examina conexiones y calcula la ruta óptima entre municipios con trasbordos reales.

RAG TARIFAS

Motor normativo

Asistente experto en títulos ATM, zonas tarifarias.

CSV ZONAS

Zonas sin RAG

Obtención de datos sin necesidad de conectar a LLM.

API TMB - iMetro

Tiempo real

Conexión con iMetro para conocer al instante la llegada del próximo metro.

Especialistas independientes coordinados por LangGraph para evitar alucinaciones.

Autor: Raúl Molina Ordóñez

Made with GAMMA

Demo en directo

[Espacio reservado para pantalla interactiva / captura Streamlit]

Autor: Raúl Molina Ordóñez

Made with GAMMA

Detalles técnicos – Preparación proyecto y API_Key

01

`git clone`

Descarga del repositorio oficial de ATeMto.

```
git clone https://github.com/rmolinao-stack/PB2-MULTIAGENTE-TARIFAS-ATM-BCN
```

02

`entorno venv`

Aislamiento con Python para control estricto de dependencias.

```
python -m venv .venv
source .venv/bin/activate
```

03

`pip install`

Instalación del core: LangChain, LangGraph, Chroma y Streamlit.

```
pip install -r requirements.txt
```

04

`config .env`

Definición de claves de API (TMB credentials y proveedor de LLM).

```
cp .env.example .env
```

Puesta en marcha directa sin contenedores obligatorios ni servicios externos pesados.

Autor: Raúl Molina Ordóñez

Made with GAMMA

Estructura del proyecto (ver README.MD)

```
PB2-MULTIAGENTE-ATM-BCN/
├── README.md
├── requirements.txt
├── .env.example
├── .gitignore
├── main.py                  # CLI: prepare / index / query / ask.
├── app_rag.py               # Asistente de tarifas - RAG en Streamlit.
├── app_agent.py             # Multiagente en Streamlit.
├── .streamlit/              # Configuración streamlit
├── assets/
│   ├── estacions.csv        # Lista de estaciones para extraer el ID necesaria para la API de TMB.
│   └── plano_ATM.jpg        # Plano para la el especialista de recorridos.
├── common/                  # Carpeta donde guardamos ficheros comunes como comfig o utils.
│   ├── config.py            # Se definen parametros generales del sistema.
│   ├── carga_api_key.py     # Carga de API key de gemini.
│   └── utils.py             # Funciones genéricas auxiliares y reutilizables.
├── data/                    # Corpus del RAG
├── entregables/             # informe final.
│   ├── informe_agent.md     # informe final de la parte de multiagente.
│   └── informe_rag.md       # informe final de la parte de rag.
├── img/                     # Imagenes varias de apoyo.
├── multiagent/              # Carpeta donde guardamos los ficheros de la capa de negocio del multiagente.
│   ├── __init__.py
│   ├── agent.py             # procesar_turno, aprobar_plan, run_demo
│   ├── graph.py             # StateGraph multiagente + MemorySaver + HITL
│   ├── llm.py               # ChatGoogleGenerativeAI
│   ├── state.py             # estado de producto (UI / resumen)
│   └── tools/
│       ├── __init__.py      # allowlist (@tool) + ejecutar_tool
│       ├── api_tiempos_tmb.py  # Tool, para más detalle ver memoría.
│       ├── info_estacion.py    # Tool, para más detalle ver memoría.
│       ├── rag_tarifas.py      # Tool, para más detalle ver memoría.
│       └── validar_plan.py     # Tool, para más detalle ver memoría.
└── vectorstore/             # Vectorstore del RAG
```

Autor: Raúl Molina Ordóñez

Made with GAMMA

Orden de ejecución estándar

PASO 1

Preparar índice

Carga el corpus, genera embeddings y recrea el índice.

```
python main.py --prepare --index --recreate-index
```

PASO 2

Lanzar el multiagente

Ejecuta el servidor Streamlit e interactúa con el multiagente.

```
streamlit run app_agent.py
```

Otros modos de ejecución

VALIDACIÓN CLI

```
--agente-interactivo
--agente-demo
```

VALIDACIÓN TOOLS

```
--test-tool-rag-tarifas
--test-tool-plano
--test-tool-zonas
--test-tool-id-estacion
--test-tool-llegada-tren
```

ASISTENTE RAG

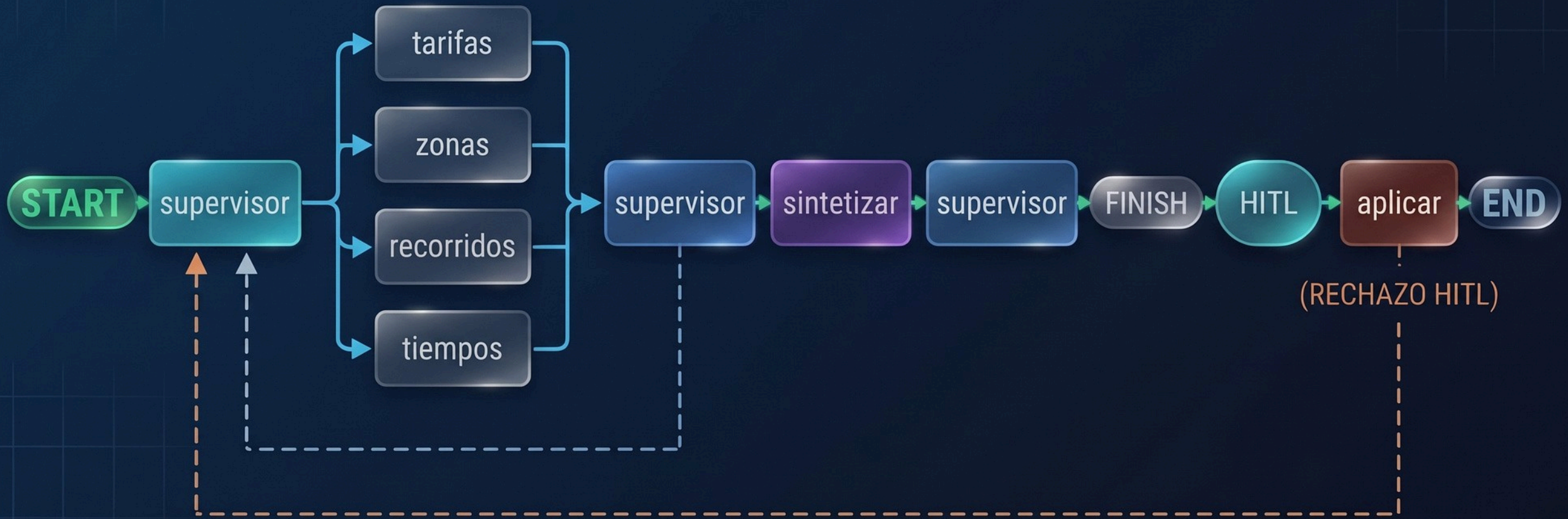
```
streamlit run app_rag.py
```

Autor: Raúl Molina Ordóñez

Made with GAMMA

ATeMto

FLUJO DE TRABAJO MULTIAGENTE



Autor: Raúl Molina Ordóñez

Made with GAMMA

Las tools que usan los especialistas

Tool	Qué hace	Tipo · Determinista
`rag_ask_tarifas`	Pregunta al RAG sobre tarifas	`RAG · No`
`zonas_poblacion_text`	Detecta poblaciones: zona, sector y tarifa metro	`Auxiliar · Sí`
`calcular_ruta_plano`	Explica en el plano cómo ir entre estaciones	`Multimodal · No`
`API_llegada_tren_estacion_tmb`	Devuelve el tiempo de los siguientes trenes (API iMetro)	`API · Sí`
`obtener_id_estacion`	Obtiene el id de estación desde un csv código-estación	`Mini-RAG / LLM · No`
`obtener_nombre_estacion_oficial`	Devuelve el nombre oficial de una estación dado su id	`Auxiliar · Sí`

Autor: Raúl Molina Ordóñez

Made with GAMMA

Muchas gracias

Para más detallas:

- Ver readme.md y memorias en carpeta de entregables.
- Contactar conmigo.

Autor: Raúl Molina Ordóñez

Made with GAMMA